Comparative Analysis of CNN and Transfer Learning Models for Pet Image Classification

Visar Sokoli

May 2025

1 Introduction

Image classification is a fundamental task in computer vision with widespread applications, including medical imaging, facial recognition, autonomous vehicles, and other everyday applications. The general goal is to correctly identify the category an image belongs to, which requires models that can effectively learn visual patterns and generalize to unseen examples. In this project, we apply image classification techniques to the Oxford-IIIT Pet Dataset, a set of images consisting of 37 different breeds of cats and dogs, each with around 200 photos. This dataset presents challenges such as fine-grained class differences, varying poses, lighting conditions, and backgrounds. These factors make it a suitable example for evaluating artificial neural network model performance.

To address this task, we explore the use of neural networks, which have shown significant success in image classification problems. Specifically, we implement and compare four models: a custom Convolutional Neural Network (CNN), and three transfer learning models — MobileNetV2, VGG16, and EfficientNetB0. The decision to use three transfer models was made based on their ability to learn complex patterns in an efficient manner of time and computation. These models are evaluated based on their classification accuracy and other metrics such as precision, recall, and F1-score. Our objective is to identify which architecture achieves the highest accuracy on this dataset while also analyzing training dynamics and generalization performance.

2 Data Description

The dataset itself contains thirty-seven different breeds of cats and dogs, with each breed containing two-hundred images, making it a balanced dataset in terms of classes. The dataset set also contains a folder of labels for each image, pet type (cat or dog) and specific breed, therefore two types of classification can be done. For this project, we will be using neural networks to predict each image of a pet's specific breed, so for each image, there will be 37 options that the model can choose between. Therefore, a random guessing model would be expected to correctly predict the breed of a pet image with an accuracy of 2.7%. However, with the use of our neural network models, we should expect their accuracy to be dramatically higher, given their extraordinary ability to learn complex patterns, like the ones that appear in household pet photos.

The pet photographs in the data are randomly selected photos from people's pets, so every photo has different backgrounds, lighting, centering, and angle at which the pet is facing. In the dataset, there are more types of dog breeds than cat breeds, therefore there are more pictures of dogs than cats. Of the 37 classes of breed, the first 10 in alphabetical order are Abyssinian, American bulldog, American pit bull terrier, basset hound, beagle, Bengal, Birman, Bombay, boxer, and British shorthair. All breed classes have 200 images of the specific breed, except the Scottish terrier and the Staffordshire bull terrier, with 199 and 191 images, respectively. This brings the

total image count for the dataset to 7390, which will be used for training and validation. A random sample of 10 images was taken to show differences in image quality, size, and appearance.

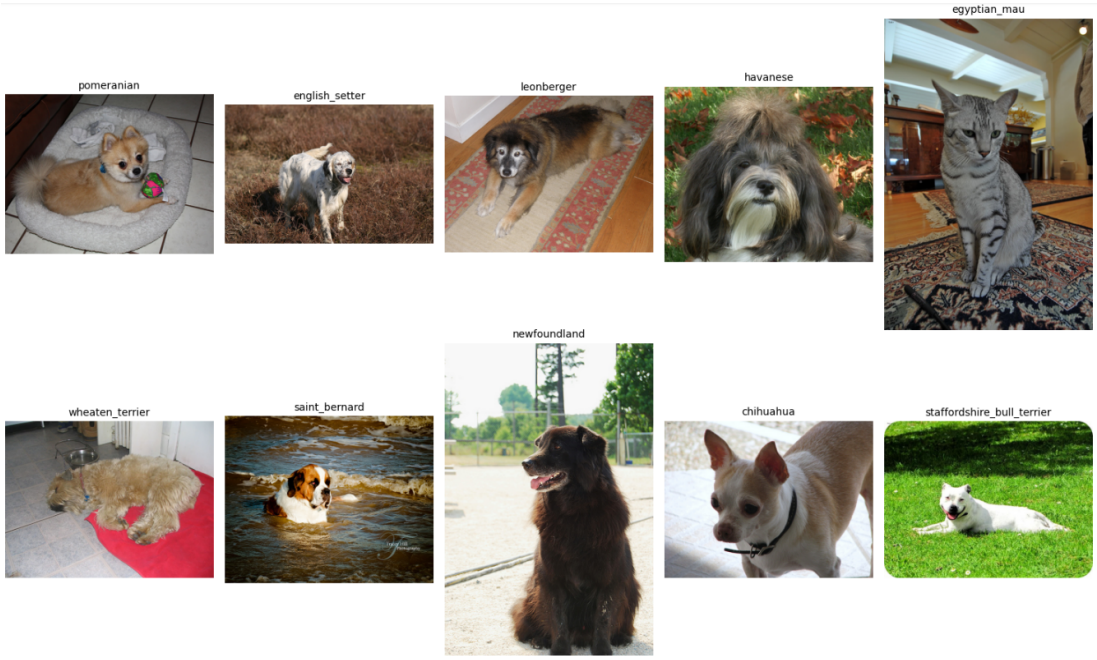

Figure 1: Sample of Pet Images

From the random sample of images we see that almost every image is a different size than the others. Therefore some data prepossessing will need to be done including a way to separate the large folder of images into sub-folders of breed images with their labels so that the neural network that we will use can prepare to train with the data.

3 Data Preprocessing

To prepare the Oxford-IIIT Pet Dataset for training neural network models, the original image files needed to be restructured into a format compatible with TensorFlow's image directory function. Initially, all pet images were stored in a single folder, with each filename containing the breed name (e.g., `Abyssinian_23.jpg`, `beagle_45.jpg`). For supervised learning tasks, it is important that the data be organized into subdirectories where each subfolder represents a single class label. To achieve this, a preprocessing script was implemented to automate the reorganization process.

The script iterates over every image in the dataset, extracts the class name from each filename, and moves the image into a corresponding class-specific directory. For example, all images with filenames starting with "Abyssinian" are moved into a folder named `abyssinian`. This structure enables TensorFlow to automatically assign class labels when loading the data, facilitating easy batching, shuffling, and splitting into training and validation sets. This preprocessing step ensures that the dataset is clean, well-organized, and ready for efficient use in neural network training.

3.1 Data Split

To prepare the Oxford-IIIT Pet Dataset for training, all images were first organized into separate class-specific directories based on pet breed names. The dataset was then loaded using a TensorFlow

utility function, which automatically assigns labels based on directory structure, resizes all images to a uniform size of 160×160 pixels, and splits the data into training and validation sets. A total of 5,912 images (80%) were used for training, while 1,478 images (20%) were reserved for validation.

To improve training efficiency and model performance, several data pipeline optimizations were applied. These included caching the dataset to avoid redundant disk reads, shuffling the training data to prevent learning order bias, and using prefetching to ensure data is prepared while the model is training, thus reducing input latency. These enhancements help streamline the training process and reduce overall computation time.

Although introducing a separate test set is a common practice for evaluating generalization, it was decided to focus solely on validation metrics for this project. This decision was made to simplify experimentation and analysis while still ensuring the models were evaluated on unseen data. We will collect different metrics on each model we test, but the main one will be accuracy, which will tell us the percentage of correct predictions each model makes.

4 Model Development

4.1 Evaluation Metrics

When evaluating classification models, especially those used for multi-class image classification tasks like the Oxford-IIIT Pet Dataset, choosing appropriate performance metrics is crucial. While accuracy remains the most intuitive and widely reported metric — representing the proportion of correctly predicted labels — it does not always tell the full story, particularly in cases where some classes may be more difficult to predict or are underrepresented.

To gain a more comprehensive understanding of each model's performance, additional metrics such as macro precision, recall, and F1-score were also used. These metrics are especially valuable for assessing how well a model can identify positive instances across all classes, rather than being skewed by the most frequent classes. For example, a model that performs well on common breeds but poorly on rarer ones may still have a high accuracy but low precision or recall. This is why macro-averaged metrics are useful — they treat all classes equally when computing performance scores, helping to reveal weaknesses that may not be visible through accuracy alone.

- **Accuracy:** The ratio of correctly predicted labels to the total number of predictions. It provides a general sense of how well the model is performing across the dataset and was the primary metric targeted for optimization.
- **Precision (Macro):** Measures the proportion of true positive predictions out of all positive predictions made by the model, averaged equally across all classes. This is important for understanding how reliable the model's predictions are for each class.
- **Recall (Macro):** Measures the proportion of actual positive instances that were correctly predicted by the model, averaged across all classes. High recall indicates that the model is good at identifying instances of each class, including less frequent ones.
- **F1-Score (Macro):** The harmonic mean of precision and recall, computed per class and then averaged. It provides a balanced measure of a model's ability to make correct predictions (precision) and to not miss positive cases (recall).

Together, these metrics offer a more nuanced picture of model performance. Accuracy tells us how often the model is right overall, but precision, recall, and F1-score help ensure that performance is strong and consistent across all pet breeds, regardless of class frequency. This multi-metric evaluation strategy reduces the risk of over-relying on a single measure and provides a fairer assessment of the model's generalization capabilities. Even though in this case most class sets are of similar size, it is

interesting to see how the models would perform on these metrics, and overall, it is good practice to use these metrics in more imbalanced datasets in the future.

4.2 Baseline Convolutional Neural Network (CNN)

Convolutional Neural Networks (CNNs) are a class of deep learning models particularly well-suited for image classification tasks. They are designed to automatically and adaptively learn spatial hierarchies of features through the use of convolutional layers, which apply filters to the input data to detect patterns such as edges, textures, and shapes. Due to their effectiveness and relative simplicity, CNNs serve as a strong starting point for image-based machine learning problems.

In this project, a custom CNN was implemented as a baseline model to evaluate and compare against more advanced architectures later in the study. The model was designed using a straightforward sequential architecture with the following components:

- **Data Augmentation Layer:** This layer applies random horizontal flips, small rotations, and zoom transformations to increase dataset variability and help prevent overfitting.
- **Rescaling Layer:** Pixel values are normalized to the $[0, 1]$ range by dividing by 255, improving model convergence.
- **Convolutional & Pooling Layers:** Three convolutional layers with increasing filter sizes (32, 64, and 128) extract low-to-high level features from the input images. Each convolution is followed by a max-pooling operation to reduce spatial dimensions and computation cost.
- **Flatten and Dense Layers:** The output is flattened and passed through a fully connected dense layer with 128 units using ReLU activation. Dropout regularization (with rates of 0.5 and 0.3) is used to reduce overfitting.
- **Output Layer:** A dense layer with 37 units (equal to the number of pet classes) uses the softmax activation function to generate class probabilities.

The model is compiled using the Adam optimizer and trained with the sparse categorical cross-entropy loss function, which is suitable for multi-class classification tasks with integer-encoded labels. Training was conducted over 15 epochs with both accuracy and loss monitored on the validation set.

This baseline CNN provides a foundational benchmark for assessing the effectiveness of more complex transfer learning models in the context of pet image classification. While training the history of the models accuracy and loss metrics on the training and validation was tracked to observe and quantify how the model was learning compared to other models used later.

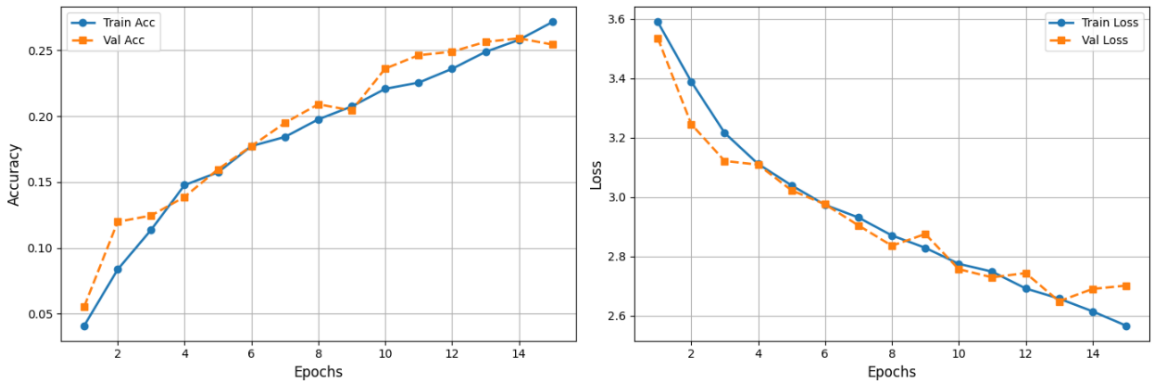

Figure 2: CNN - Accuracy & Loss Curves

Table 1: CNN Performance

Model	Accuracy	Precision	Recall	F1-score
CNN	0.254	0.273	0.259	0.239

Despite the simplicity and interpretability of the baseline CNN model, its performance on the dataset was limited, achieving only around 25% validation accuracy after 15 epochs and 2 minutes of training. Several factors may have contributed to this result. First, the dataset includes 37 different classes, many of which represent visually similar breeds or species, making it a highly challenging classification problem—especially for a shallow network like the one implemented. The model may not have had sufficient capacity (i.e., depth or number of trainable parameters) to effectively learn the complex features needed to distinguish between fine-grained pet categories. Additionally, while data augmentation was applied, the size of the training data and the relatively small input dimensions (160x160) may have further limited the model's ability to generalize. The learning curves at a first glance, give us the impression that the model is generalizing well to the unseen data due to the similarity between training and validation accuracy curves, but from the very low accuracy it can be concluded that the model may be underfitting to the data. These findings highlight the limitations of a simple CNN for fine-grained, multi-class image classification tasks, and reinforce the need to explore more powerful architectures such as pretrained models.

4.3 Transfer Learning with MobileNetV2

MobileNetV2 is a lightweight convolutional neural network architecture developed by Google, designed for efficiency on devices with limited computational resources. For this project, it was used as a transfer learning model to improve upon the baseline CNN performance by leveraging rich feature representations learned from the large-scale ImageNet dataset.

The model was constructed with the following core components:

- **Base Model:** The MobileNetV2 architecture was used without its top classification layers (`include_top=False`) and initialized with pre-trained weights from ImageNet.
- **Frozen Layers:** The base model's weights were frozen during training to preserve the generalized image features it had already learned.
- **Preprocessing:** A `Rescaling` layer normalized pixel values to the [0, 1] range to match the training conditions of the pre-trained model.
- **Classification Head:** A global average pooling layer was followed by a dense layer with 128 units and ReLU activation, then a dropout layer (rate = 0.3) for regularization.
- **Output Layer:** A dense layer with `softmax` activation was used to classify images into 37 pet classes.

The model was compiled using the Adam optimizer and trained with the sparse categorical cross-entropy loss function, suitable for multi-class classification with integer-encoded labels. To enhance training stability and avoid overfitting, an early stopping callback was implemented to monitor validation loss and restore the best-performing weights after three epochs of no improvement.

This approach provided a strong benchmark by efficiently transferring knowledge from a much larger dataset. Its small size and optimized structure made it particularly effective for extracting meaningful patterns from the Pet Image Dataset, without requiring extensive training on the current dataset. Again while training, the history of the models accuracy and loss metrics on the training and validation data was tracked to observe and quantify how the model was learning compared to other models.

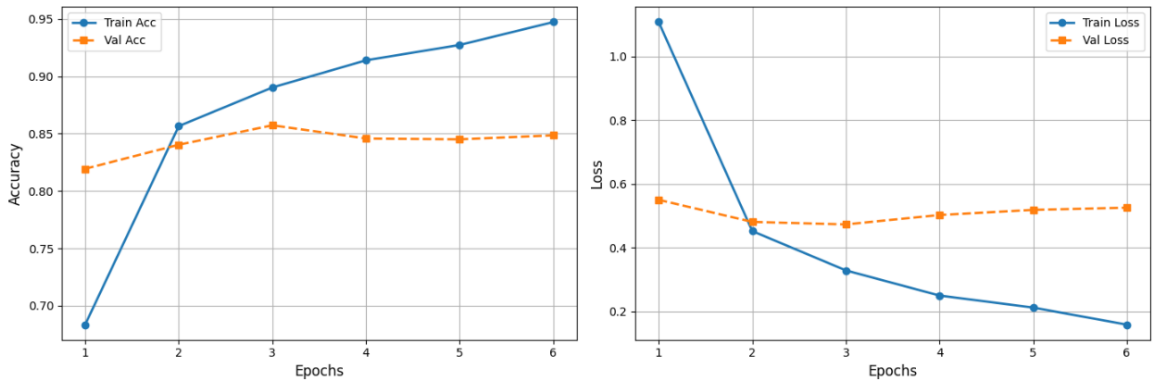

Figure 3: MobilenetV2 - Accuracy & Loss Curves

Table 2: MobileNetV2 Performance

Model	Accuracy	Precision	Recall	F1-score
MNV2	0.857	0.865	0.858	0.857

Right from the start we that this transfer learning model drastically outperformed the baseline CNN on every metric, which was expected due to the advanced nature of this model and simplicity of the CNN that was used. Due to the introduction of the early stopping callback, the MobileNetV2 model ran for 6 epochs and 45 seconds of training and achieved a validation accuracy of 85.7%. From the curves above, we see that the model achieved its maximum validation accuracy at the third epoch and was stopped after three more epochs of no improvement of validation loss due to the early stopping function having a patience of 3.

MobileNetV2 significantly outperformed the baseline CNN due to the advantages provided by transfer learning. While the baseline CNN was trained from scratch on the Pet Dataset, MobileNetV2 benefited from pre-trained weights learned on the massive and diverse ImageNet dataset. These weights allow MobileNetV2 to start with a strong understanding of general image features such as edges, textures, and shapes, which are transferable to many computer vision tasks. In contrast, the CNN must learn all patterns from scratch using only the limited pet image data, often leading to underfitting or poor generalization. Additionally, MobileNetV2's architecture is highly optimized for efficient feature extraction with fewer parameters, enabling it to learn more complex representations without overfitting. This makes transfer learning models like MobileNetV2 particularly powerful when working with moderate-sized datasets where training a deep network from scratch would be less effective.

4.4 Transfer Learning with VGG16

The third model implemented was VGG16, another widely-used convolutional neural network that has been pre-trained on the ImageNet dataset. VGG16 is known for its simplicity and depth, consisting of 16 layers and using small 3×3 convolutional filters throughout. Like MobileNetV2, VGG16 was employed as a transfer learning model, where the base layers were frozen and a custom classification head was added. This approach allowed the model to reuse previously learned features while being fine-tuned for the Pet Dataset.

- **Base Model:** Used the pre-trained VGG16 model with `include_top=False` to remove the original classifier.

- **Frozen Layers:** All layers in the base model were frozen to preserve ImageNet features.
- **Preprocessing:** Input images were preprocessed using `vgg16.preprocess_input()` to match the original training conditions.
- **Classification Head:** A global average pooling layer was followed by a dropout layer (rate = 0.3) for regularization.
- **Output Layer:** A dense layer with `softmax` activation was used to classify images into 37 pet classes.

The model was compiled using the Adam optimizer and trained with the sparse categorical cross-entropy loss function, suitable for multi-class classification with integer-encoded labels. To enhance training stability and avoid overfitting, an early stopping callback was implemented to monitor validation loss and restore the best-performing weights after three epochs of no improvement.

VGG16 provided a robust benchmark due to its proven ability to extract rich visual features. Although it is larger and computationally heavier than MobileNetV2, its layered design helped in capturing intricate patterns in pet images. Freezing the convolutional base ensured training efficiency and reduced the risk of overfitting, especially given the dataset size. By fine-tuning only the top layers, the model was able to adapt to the specific classification task while benefiting from the generalization power of pre-learned ImageNet representations. While training, the history of the models accuracy and loss metrics on the training and validation data was tracked to observe and quantify how the model was learning compared to other models.

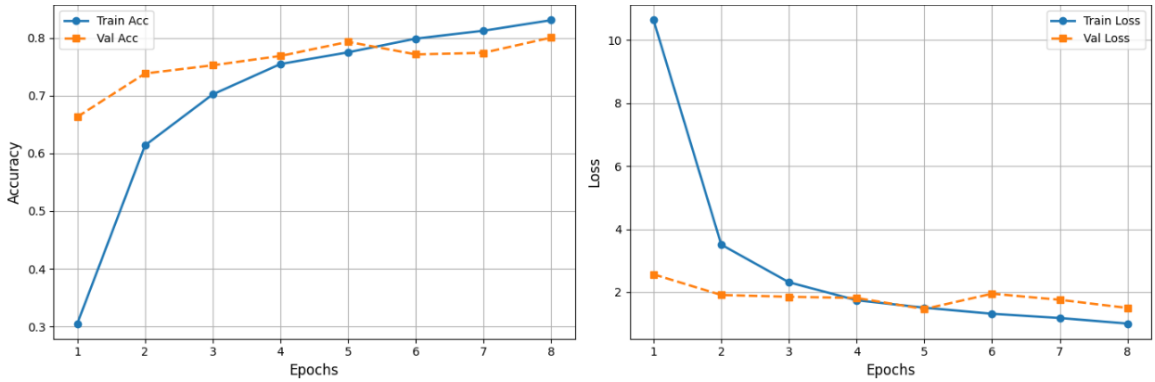

Figure 4: VGG16 - Accuracy & Loss Curves

Table 3: VGG16 Performance

Model	Accuracy	Precision	Recall	F1-score
VGG16	0.793	0.795	0.793	0.789

Immediately, we see that this transfer learning model drastically outperformed the baseline CNN on every metric, but did not outperform the MobileNet model used previously. Due to the introduction of the early stopping callback, the VGG16 model ran for 8 epochs and 4 minutes of training and achieved a validation accuracy of 79.3%. From the curves above, we see that the model achieved its maximum validation accuracy at the fifth epoch and was stopped after three more epochs of no improvement of validation loss due to the early stopping function having a patience of 3. Another to note, is that the model seemed to generalize better to new data then previous models because

of how close the accuracy curves are, but the model struggled to achieve the performance of the MobileNetV2 model used previously.

VGG16 outperformed the baseline CNN due to its deeper architecture and the advantage of transfer learning. While the CNN was trained from scratch on a relatively small dataset, VGG16 leveraged rich, pre-trained features learned from millions of diverse images in ImageNet, allowing it to generalize better and detect more complex patterns. However, despite its solid performance, VGG16 was outperformed by MobileNetV2. This difference stems from architectural efficiency: MobileNetV2 is a more modern network optimized for speed and accuracy using techniques like depthwise separable convolutions and inverted residuals. These innovations allow it to capture relevant features with fewer parameters and lower computational cost. In contrast, VGG16, although powerful, is a heavier model with more redundant layers and lacks these efficiency mechanisms, which can limit its performance when fine-tuning on smaller datasets. Thus, while both transfer learning models significantly outperformed the baseline CNN, MobileNetV2 demonstrated a more effective balance of generalization and efficiency for this classification task.

4.5 Transfer Learning with EfficientNetB0

The fourth model implemented was EfficientNetV2, a state-of-the-art convolutional neural network architecture that builds upon the original EfficientNet design with improved training speed and parameter efficiency. EfficientNetV2 is known for using compound scaling to balance network depth, width, and resolution while incorporating advanced building blocks like fused MBConv layers. Like MobileNetV2 and VGG16, EfficientNetV2 was employed as a transfer learning model, utilizing pre-trained weights from the ImageNet dataset. The base model's layers were frozen, and a custom classification head was added to adapt the network to the Oxford Pet Dataset.

- **Base Model:** Used the pre-trained EfficientNetB0 model with `include_top=False` to remove the original classifier.
- **Frozen Layers:** All layers in the base model were frozen to preserve ImageNet features.
- **Preprocessing:** Input images were preprocessed using `efficientnet.preprocess_input()` to match the original training conditions.
- **Classification Head:** A global average pooling layer was followed by a dropout layer (rate = 0.2) for regularization.
- **Output Layer:** A dense layer with `softmax` activation was used to classify images into 37 pet classes.

The model was compiled using the Adam optimizer and trained with the sparse categorical cross-entropy loss function, suitable for multi-class classification with integer-encoded labels. To enhance training stability and avoid overfitting, an early stopping callback was implemented to monitor validation loss and restore the best-performing weights after three epochs of no improvement.

EfficientNetV2 served as a powerful benchmark due to its cutting-edge architecture, which achieves high accuracy with relatively fewer parameters and faster convergence compared to older models. While it shares the lightweight design philosophy of MobileNetV2, EfficientNetV2 benefits from more sophisticated layer structures and optimization techniques, allowing it to capture detailed image features even more effectively. By freezing the convolutional base, training was made more efficient and the risk of overfitting was minimized. Fine-tuning only the upper layers allowed the model to specialize in distinguishing between the various pet breeds while leveraging the general visual representations learned from ImageNet. Throughout training, the model's performance was tracked using accuracy and loss metrics to evaluate its learning progress in comparison to the other models.

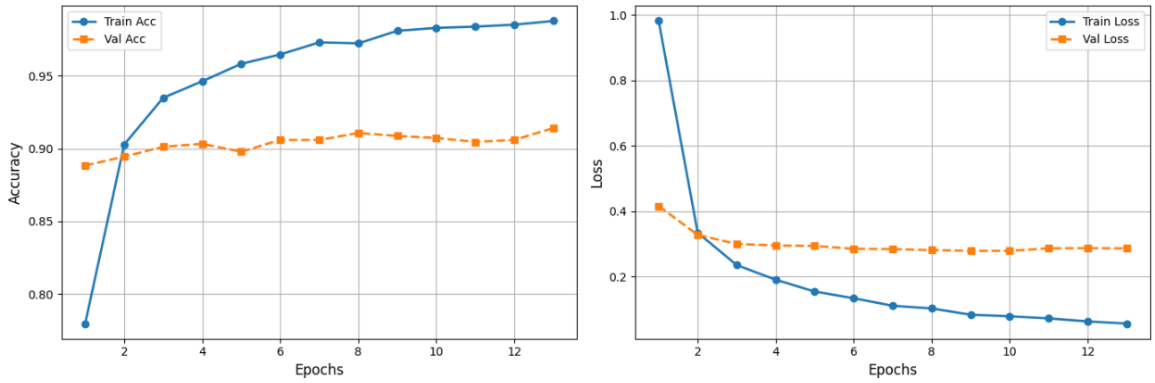

Figure 5: EfficientNetB0 - Accuracy & Loss Curves

Table 4: EfficientNetB0 Performance

Model	Accuracy	Precision	Recall	F1-score
ENB0	0.907	0.907	0.908	0.906

For this last model, we see that it drastically outperformed the baseline CNN on every metric, and also outperformed the previous 2 transfer learning models by a sizable amount. Due to the introduction of the early stopping callback, the EfficientNetB0 model ran for 13 epochs and 2 minutes of training and achieved a validation accuracy of 90.7%. From the curves above, we see that the model achieved its maximum validation accuracy at the thirteenth epoch and was stopped there because of no improvement in validation loss due to the early stopping function having a patience of 3.

EfficientNetB0 demonstrated the strongest performance on the dataset, significantly outperforming both the custom-built CNN and the other transfer learning models, MobileNetV2 and VGG16. Unlike the baseline CNN, which had to learn all visual features from scratch, EfficientNetB0 leveraged rich, pre-trained representations from ImageNet, giving it a strong head start in identifying pet-specific features. Compared to MobileNetV2 and VGG16, EfficientNetB0 benefits from a more advanced design that uses compound scaling to efficiently balance the model's depth, width, and input resolution. This enables it to achieve high accuracy with fewer parameters and faster training. While MobileNetV2 focuses on lightweight performance and VGG16 relies on deep but computationally expensive layers, EfficientNetB0 combines efficiency and depth in a much more optimized way. Its use of fused MBConv layers and improved training dynamics allows it to extract fine-grained patterns in pet images with greater precision. These architectural advantages help explain why EfficientNetB0 was able to outperform all other models by a notable margin on the Pet Dataset.

5 Conclusion

This project explored the development and evaluation of various deep learning models for the task of image classification using the Oxford-IIIT Pet Dataset. Starting with a baseline convolutional neural network (CNN), we incrementally applied more advanced transfer learning techniques, incorporating pre-trained architectures such as MobileNetV2, VGG16, and EfficientNetB0. Each model was trained and validated using consistent procedures, with performance assessed through accuracy, macro precision, recall, and F1-score.

The following table summarizes the performance of each model across the selected evaluation metrics:

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
Baseline CNN	0.254	0.273	0.259	0.239
MobileNetV2	0.857	0.865	0.858	0.857
VGG16	0.793	0.795	0.793	0.789
EfficientNetB0	**0.907**	**0.907**	**0.908**	**0.906**

Table 5: Performance comparison of all tested models.

Through this experimentation, we observed that simple CNN architectures were limited in their ability to generalize and extract deep visual features. In contrast, transfer learning models, particularly those leveraging pre-trained weights from ImageNet, significantly outperformed the baseline. MobileNetV2 demonstrated an excellent balance between performance and efficiency, while EfficientNetB0 achieved the highest performance overall due to its compound scaling strategy and modern architecture.

This comparative study highlights the value of transfer learning in image classification tasks, especially when working with relatively small datasets. The use of macro-averaged evaluation metrics further ensured that model performance was not biased toward dominant classes, offering a fair and thorough assessment.

Ultimately, this project reinforces the effectiveness of using pre-trained deep learning models as a foundation for specialized classification tasks. It also showcases how careful metric selection and model comparison can guide informed decisions in real-world applications, especially in domains where balanced performance across classes is critical.

Appendix

Dataset

The dataset used in this analysis is the **Oxford-IIIT Pet Dataset**, available at: `https://www.robots.ox.ac.uk/~vgg/data/pets/`

Implementation

All coding and plots were performed using **Python**. The following key libraries and functions were used frequently throughout the analysis:

- `pandas` – For data manipulation and preprocessing.
- `numpy` – For numerical operations.
- `matplotlib` – For data visualization and plotting.
- `sklearn` – For calculating evaluation metrics
- `tenserflow & keras` – Used for neural network model definition, data augmentation, callbacks like EarlyStopping, and model compilation.
- `os` – Used to manage directories, file paths, or organize datasets if working locally.

Model Parameters

The parameters for the neural network models used in this paper are as follows:

- **Baseline CNN:**
  - `Trainable Parameters` = 405,861
  - `Convolution Layers` = 3
  - `Dense Layers` = 2
  - `Dropout Rates` = 0.5 & 0.3
  - `Model Activation` = 'ReLU'
  - `Epochs` = 15
  - `Batch Size` = 16
  - `Optimizer` = 'Adam'
- **MobileNetV2:**
  - `Trainable Parameters` = 185,989
  - `Global Pooling` = 'GlobalAveragePooling2D'
  - `Dense Layers` = 2
  - `Dropout Rates` = 0.3
  - `Model Activation` = 'ReLU'
  - `Epochs` = 15 (with early stopping)
  - `Batch Size` = 16
  - `Optimizer` = 'Adam'

- **VGG16 & EffiecientNetB0:**
  - `Trainable Parameters` = 9,454
  - `Global Pooling` = 'GlobalAveragePooling2D'
  - `Dense Layers` = 1
  - `Dropout Rates` = 0.3
  - `Model Activation` = 'ReLU'
  - `Epochs` = 15 (with early stopping)
  - `Batch Size` = 16
  - `Optimizer` = 'Adam'